

JavaScript

Day-Long Workshop — Basic to Advanced, Step by Step

National Institute of Youth Development (NIYD)

Every example is complete — copy, paste, and run!

Today's Journey — 8 Parts



1

Basics

Variables, types,
strings, operators

2

Decisions & Loops

if / else, switch,
for, while

3

Functions

Reusable code
blocks

4

Arrays

Lists: map, filter,
reduce, sort

5

Objects

Key : value data,
methods, this

6

DOM & Events

Control the page,
clicks, forms

7

Modern JS (ES6)

Template strings,
spread, classes

8

Async & Errors

fetch, await,
try / catch

How to try every example: Open Chrome → press F12 → click "Console" → paste the code → press Enter

What is JavaScript?

index.html

```
<!DOCTYPE html>
<html>
<body>
  <h1>My First Page</h1>

  <script>
    alert("Hello NIYD!");
    console.log("JavaScript is working!");
  </script>

</body>
</html>
```

- HTML makes the page. CSS makes it beautiful.
- **JavaScript makes it DO things.**
- JS code goes inside <script> ... </script>
- Save this as index.html and double-click it

What you will see

A popup box appears:

"Hello NIYD!"

And in the console:

JavaScript is working!

3 Ways to Add JavaScript

index.html

```
<!-- 1. Inline — avoid this -->
<button onclick="alert('Hi')">Hi</button>

<!-- 2. Internal — small pages -->
<script>
  console.log("I live inside HTML");
</script>

<!-- 3. External — the best way -->
<script src="app.js"></script>

<!-- app.js is a separate file
     with only JavaScript inside -->
```

- Three ways to connect JS to a page
- External file keeps code clean & reusable
- **Rule: put <script> just before </body>**
- One app.js can be used by many pages

Good to know

All three ways work the same.
Professionals use way 3:
a separate .js file.

JavaScript

PART 1 OF 8

Basics

We are learning about:

Variables · Data types · Strings · Numbers · Operators

Variables — let & const



variables.js

```
// let = value CAN change
let age = 25;
age = 26;

// const = value can NOT change
const name = "Rahim";

console.log(name, age);

// name = "Karim";  <- this line
// would give an error!
```

- **A variable is a box that stores a value**
- let → you can change it later
- const → fixed, cannot be changed
- Tip: always start with const

Output

```
Rahim 26
```

Why Not var?



var-problem.js

```
// var is the OLD way (before 2015)
var x = 1;
var x = 2;   // re-declare? no error!
console.log(x);

// let protects you:
let y = 1;
// let y = 2;  <- Error! Already
//               declared. Good!

if (true) {
  let inside = "secret";
}
// console.log(inside); <- Error!
// let lives only inside its { }
```

- var is old — it allows silent mistakes
- let / const warn you about re-declaring
- let / const live only inside their { }
- **In new code: const first, let second, var never**

Output

```
2
```

Data Types



types.js

```
const city = "Dhaka";       // String (text)
const price = 99.5;         // Number
const isOpen = true;        // Boolean
let winner;                 // Undefined
const empty = null;         // Null (empty)

console.log(typeof city);
console.log(typeof price);
console.log(typeof isOpen);
console.log(typeof winner);
```

- String = text, always in quotes " "
- Number = 42, 3.14 — no quotes
- Boolean = only true or false
- typeof tells you the type of anything

Output

```
string
number
boolean
undefined
```

Strings — Working with Text



strings.js

```
const msg = "JavaScript is fun";

console.log(msg.length);
console.log(msg.toUpperCase());
console.log(msg.toLowerCase());
console.log(msg.includes("fun"));
console.log(msg.slice(0, 10));
console.log(msg.replace("fun", "easy"));
```

- .length → how many characters
- .toUpperCase() → ALL CAPITALS
- .includes() → is this text inside?
- .slice(0, 10) → cut a piece (from 0 to 10)

Output

```
17
JAVASCRIPT IS FUN
javascript is fun
true
JavaScript
JavaScript is easy
```

Numbers & Math

numbers.js

```
const price = 99.567;

console.log(price.toFixed(2));

console.log(Math.round(4.6));   // nearest
console.log(Math.floor(4.6));   // down
console.log(Math.ceil(4.2));    // up
console.log(Math.max(3, 7, 2));

// Random number 0 to 1:
console.log(Math.random());
```

- .toFixed(2) → keep 2 decimal places
- Math.round / floor / ceil → rounding
- Math.max / Math.min → biggest / smallest
- Math.random() → different every time!

Output

```
"99.57"
5
4
5
7
0.7231... (random!)
```

Operators — Math



math.js

```
const a = 10, b = 4;

console.log(a + b);    // add
console.log(a - b);    // subtract
console.log(a * b);    // multiply
console.log(a / b);    // divide
console.log(a % b);    // remainder

// Shortcuts:
let score = 5;
score += 10;           // score = score + 10
score++;               // add 1
console.log(score);
```

- + - * / work like a calculator
- % gives the remainder (10 % 4 = 2)
- += adds to the current value
- ++ adds exactly 1 (great for counters)

Output

```
14
6
40
2.5
2
16
```

Compare & Combine

compare.js

```
const age = 20;

console.log(age > 18);
console.log(age === 20);
console.log(age !== 18);

// Combine conditions:
console.log(age > 18 && age < 30); // AND
console.log(age < 10 || age > 15); // OR

// The famous trap:
console.log(5 == "5");   // value only
console.log(5 === "5");  // value + type
```

- Comparing always gives true or false
- && = both must be true (AND)
- || = at least one is true (OR)
- **Always use === — it is safer than ==**

Output

```
true
true
true
true
true
true
false
```

PART 2 OF 8

Decisions & Loops

if / else · switch · ternary · for · while · break

if / else — Two Roads



weather.js

```
const temperature = 35;

if (temperature > 30) {
  console.log("It is hot today!");
  console.log("Drink water.");
} else {
  console.log("Nice weather.");
}
```

- if (condition) ⟶ runs when true
- else ⟶ runs when false
- Only ONE of the two roads runs
- Change 35 to 20 and run again!

Output

```
It is hot today!
Drink water.
```

else if — The Grading Ladder

grades.js

```
const mark = 75;
let grade;

if (mark >= 80)       grade = "A+";
else if (mark >= 70)  grade = "A";
else if (mark >= 60)  grade = "B";
else if (mark >= 50)  grade = "C";
else                  grade = "F";

console.log("Your grade:", grade);
```

- Checks from top to bottom
- Stops at the FIRST true condition
- else at the end catches everything left
- Try marks 85, 65, 40 — watch it change

Output

```
Your grade: A
```

switch — Many Fixed Choices



days.js

```
const day = "Friday";

switch (day) {
  case "Friday":
    console.log("Weekend starts!");
    break;
  case "Sunday":
    console.log("Back to work.");
    break;
  default:
    console.log("A normal day.");
}
```

- Good when one value has many fixed options
- case = one possible value
- **break is important — without it, cases leak!**
- default = like else, catches the rest

Output

```
Weekend starts!
```

Ternary — if / else in One Line



ternary.js

```
const mark = 75;

// condition ? ifTrue : ifFalse
const result = mark >= 60 ? "Pass" : "Fail";
console.log(result);

const age = 16;
const ticket = age < 18 ? "Child" : "Adult";
console.log(ticket);

const price = 500;
console.log(price > 300 ? "Costly" : "Cheap");
```

- A short way to choose between 2 values
- Read it as: "is it true ? yes-value : no-value"
- Perfect for simple choices
- For 3+ roads, use if / else if instead

Output

```
Pass
Child
Costly
```

Truthy & Falsy



truthy.js

```
const name = "";

if (name) {
  console.log("Hello, " + name);
} else {
  console.log("Name is empty!");
}

// Only 6 values act like false:
// false   0   ""   null
// undefined     NaN
// EVERYTHING else acts like true
```

- JS can treat any value as true / false
- Empty text "" acts like false
- 0 acts like false, any other number is true
- Great for checking "did the user type something?"

Output

```
Name is empty!
```

for — Count and Repeat



for.js

```
// start ; condition ; step
for (let i = 1; i <= 3; i++) {
  console.log("Number:", i);
}

// Count down too!
for (let i = 3; i >= 1; i--) {
  console.log("Countdown", i);
}
console.log("Go!");
```

- i = 1 ⟶ start here
- i <= 3 ⟶ keep going while this is true
- i++ ⟶ add 1 after every round
- Change 3 to 100 — that's the power!

Output

```
Number: 1
Number: 2
Number: 3
Countdown 3
Countdown 2
Countdown 1
Go!
```

while — Repeat Until Done



while.js

```
let count = 1;

while (count <= 3) {
  console.log("While:", count);
  count++;    // never forget this!
}

// do...while runs at least ONCE:
let n = 10;
do {
  console.log("I ran once anyway");
} while (n < 5);
```

- while ⟶ use when you don't know how many times
- **Forgetting count++ = infinite loop! (freeze)**
- do...while ⟶ checks AFTER running once
- for = known count, while = unknown count

Output

```
While: 1
While: 2
While: 3
I ran once anyway
```

for...of & for...in

forof.js

```
// for...of → values of a list
const fruits = ["Mango", "Guava", "Lichu"];

for (const fruit of fruits) {
  console.log(fruit);
}

// for...in → keys of an object
const student = { name: "Ayesha", age: 21 };

for (const key in student) {
  console.log(key, "=", student[key]);
}
```

- **for...of ⟶ easiest way to loop a list**
- No counting, no i++, no mistakes
- for...in ⟶ walks the keys of an object
- Memory trick: of = values, in = keys

Output

```
Mango
Guava
Lichu
name = Ayesha
age = 21
```

break & continue

control.js

```
for (let i = 1; i <= 10; i++) {

  if (i === 3) {
    continue;    // skip number 3
  }

  if (i === 6) {
    break;       // stop completely at 6
  }

  console.log(i);
}
```

- continue → skip this round, go to next
- break → leave the loop completely
- 3 is missing (skipped)
- 5 is the last (6 stopped everything)

Output

```
1
2
4
5
```

Practice — Times Table



table.js

```
const number = 5;

for (let i = 1; i <= 10; i++) {
  console.log(`${number} x ${i} = ${number * i}`);
}

// Challenge: change number to 7
// Challenge 2: print tables 1 to 5
//   (hint: a loop inside a loop!)
```

- A loop + math = instant times table
- `${ }` puts values inside text
- (full lesson on this in Part 7!)
- **Try the challenges before moving on**

Output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50
```

PART 3 OF 8

Functions

Declarations · Arrow functions · Parameters · Return · Scope

Functions — Write Once, Use Forever



greet.js

```
// Define the machine once:
function greet(name) {
  console.log("Hello, " + name + "!");
}

// Use it as many times as you want:
greet("Rahim");
greet("Karim");
greet("Ayesha");
```

- function name(input) { work }
- Define once → call many times
- name is a parameter (the input)
- No more copy-pasting the same code!

Output

```
Hello, Rahim!
Hello, Karim!
Hello, Ayesha!
```

Arrow Functions — The Modern Way



arrows.js

```
// Function stored in a variable:
const add = function (a, b) {
  return a + b;
};

// Same thing with an arrow =>
const multiply = (a, b) => a * b;

// One input? Skip the ( )
const square = n => n * n;

console.log(add(2, 3));
console.log(multiply(4, 5));
console.log(square(6));
```

- => is just a shorter way to write a function
- One line? The result returns automatically
- You will see arrows everywhere in modern JS
- All three forms here do the same kind of job

Output

```
5
20
36
```

return & Default Values



return.js

```
// return sends the result back:
function total(price, qty) {
  return price * qty;
}

const bill = total(50, 3);
console.log("Bill:", bill);

// Default value if nothing is given:
function makeTea(sugar = 2) {
  return `Tea with ${sugar} spoons`;
}

console.log(makeTea());
console.log(makeTea(5));
```

- console.log = show it, return = give it back
- A returned value can be stored and reused
- sugar = 2 ⟶ used when no value is passed
- After return, the function stops

Output

```
Bill: 150
Tea with 2 spoons
Tea with 5 spoons
```

Scope — Where Variables Live



scope.js

```
const appName = "MyApp";    // global

function showUser() {
  const user = "Rahim";     // local
  console.log(appName, user);
}

showUser();
console.log(appName);

// console.log(user);  <- Error!
// user only lives INSIDE the function
```

- Global = made outside, visible everywhere
- Local = made inside { }, visible only there
- Inside can see outside — not the reverse
- Small scopes = fewer accidents

Output

```
MyApp Rahim
MyApp
```

PART 4 OF 8

Arrays

push / pop · search · map · filter · reduce · sort

Arrays — A List of Values



arrays.js

```
const fruits = ["Mango", "Guava", "Lichu"];

console.log(fruits[0]);   // first
console.log(fruits[2]);   // third
console.log(fruits.length);

// Change one item:
fruits[1] = "Banana";
console.log(fruits);
```

- An array is a list: [item, item, item]
- **Counting starts at 0, not 1!**
- .length ⟶ how many items
- fruits[1] = ... ⟶ replace an item

Output

```
Mango
Lichu
3
["Mango", "Banana",
 "Lichu"]
```

Add & Remove Items

addremove.js

```
const list = ["b", "c"];

list.push("d");       // add to END
list.unshift("a");    // add to START
console.log(list);

list.pop();           // remove LAST
list.shift();         // remove FIRST
console.log(list);
```

- push / pop → work at the END
- unshift / shift → work at the START
- Memory trick: push = put, pop = pull out
- These change the original array

Output

```
["a", "b", "c", "d"]
["b", "c"]
```

Searching in Arrays



search.js

```
const nums = [10, 20, 30, 40];

console.log(nums.includes(30));
console.log(nums.includes(99));

console.log(nums.indexOf(20));

// find → first item matching a rule:
const found = nums.find(n => n > 25);
console.log(found);
```

- includes ⟶ is it there? true / false
- indexOf ⟶ at which position? (-1 = not found)
- find ⟶ give me the first match
- find uses a small arrow function as the rule

Output

```
true
false
1
30
```

map & filter — The Power Tools



mapfilter.js

```
const prices = [100, 200, 300];

// map → CHANGE every item
const doubled = prices.map(p => p * 2);
console.log(doubled);

// filter → KEEP some items
const cheap = prices.filter(p => p < 250);
console.log(cheap);

// Original is never touched:
console.log(prices);
```

- **map = change every item ⟶ new array**
- **filter = keep matching items ⟶ new array**
- The original array stays safe
- These two run the modern web — learn them well

Output

```
[200, 400, 600]
[100, 200]
[100, 200, 300]
```

reduce — Many Values → One



reduce.js

```
const bill = [120, 80, 250];

// Add everything together:
const total = bill.reduce(
  (sum, item) => sum + item,
  0   // start counting from 0
);

console.log("Total:", total);

// sum grows: 0+120 -> 120+80
//            -> 200+250 -> 450
```

- reduce squeezes a list into ONE value
- sum = the running total so far
- 0 = the starting value
- Perfect for totals, averages, counters

Output

```
Total: 450
```

sort, reverse & join



sort.js

```
const names = ["Rahim", "Ayesha", "Karim"];

names.sort();
console.log(names);

names.reverse();
console.log(names);

// join → array becomes one text:
console.log(names.join(" - "));

// Numbers need a helper to sort:
console.log([30, 4, 100].sort((a, b) => a - b));
```

- sort() ⟶ alphabetical order (A to Z)
- reverse() ⟶ flip the order
- join(" - ") ⟶ glue items into one string
- Numbers: sort((a, b) => a - b) — remember this!

Output

```
["Ayesha", "Karim", "Rahim"]
["Rahim", "Karim", "Ayesha"]
Rahim - Karim - Ayesha
[4, 30, 100]
```

PART 5 OF 8

Objects

Key : value · Methods · this · Arrays of objects

Objects — Facts About One Thing



phone.js

```
const phone = {
  brand: "Samsung",
  model: "A54",
  price: 45000
};

console.log(phone.brand);
console.log(phone["price"]);

phone.price = 42000;      // update
phone.color = "Black";    // add new
console.log(phone);
```

- Each fact is a pair → key: value
- Read with a dot: phone.brand
- phone.price = ... → update a value
- A new key? Just assign it — it appears!

Output

```
Samsung
45000
{brand: "Samsung",
 model: "A54",
 price: 42000,
 color: "Black"}
```

Methods & this



account.js

```
const account = {
  owner: "Rahim",
  balance: 5000,

  deposit(amount) {
    this.balance += amount;
    console.log(this.owner,
      "now has", this.balance);
  }
};

account.deposit(2000);
account.deposit(500);
```

- A function inside an object = a method
- **this = "the object I live inside"**
- this.balance ⟶ this account's balance
- The object carries data + actions together

Output

```
Rahim now has 7000
Rahim now has 7500
```

Object.keys, values & Nesting



student.js

```
const student = {
  name: "Ayesha",
  dept: "CSE",
  address: {
    city: "Dhaka",
    area: "Mirpur"
  }
};

console.log(Object.keys(student));

// Objects inside objects — use 2 dots:
console.log(student.address.city);
```

- Object.keys() → list of all keys
- Object.values() → list of all values
- Objects can nest (an object inside an object)
- Go deeper with more dots: a.b.c

Output

```
["name", "dept",
 "address"]
Dhaka
```

Arrays of Objects — Real Data!



class.js

```
const students = [
  { name: "Rahim",  mark: 85 },
  { name: "Karim",  mark: 55 },
  { name: "Ayesha", mark: 92 }
];

for (const s of students) {
  console.log(s.name, "got", s.mark);
}

const passed = students.filter(s => s.mark >= 60);
console.log(passed.length, "passed");
```

- **This is the shape of ALL real data (JSON)**
- A list, where each item is an object
- Loop it, filter it, map it — all tools work
- Facebook posts, products, users — all like this

Output

```
Rahim got 85
Karim got 55
Ayesha got 92
2 passed
```

PART 6 OF 8

DOM & Events

Select · Change · Create · Clicks · Typing · Forms · Mini project

DOM — Find Things on the Page



select.html

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="title">Hello</h1>
  <p class="info">First</p>
  <p class="info">Second</p>

  <script>
    const h1 =
      document.getElementById("title");
    const all =
      document.querySelectorAll("p");

    console.log(h1.textContent);
    console.log(all.length);
  </script>
</body>
</html>
```

- DOM = the page, seen from JavaScript
- getElementById ⟶ find by id (one element)
- querySelectorAll ⟶ find all matches
- Save as select.html ⟶ open ⟶ press F12

Output

```
Hello
2
```

DOM — Change Text & Style

change.html

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="title">Hello</h1>

  <script>
    const h1 =
      document.getElementById("title");

    h1.textContent = "Welcome to NIYD!";
    h1.style.color = "white";
    h1.style.background = "tomato";
    h1.style.padding = "10px";
  </script>
</body>
</html>
```

- Step 1: find it. Step 2: change it.
- .textContent → change the words
- .style.color / background → change CSS
- Anything CSS can do, JS can set

What you will see

The heading becomes "Welcome to NIYD!" — white text in a red box.

DOM — Add Classes & New Elements



create.html

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .highlight { background: yellow; }
  </style>
</head>
<body>
  <p id="msg">Watch me change</p>
  <ul id="list"></ul>

  <script>
    document.getElementById("msg")
      .classList.add("highlight");

    const li = document.createElement("li");
    li.textContent = "New item";
    document.getElementById("list")
      .appendChild(li);
  </script>
</body>
</html>
```

- classList.add → switch a CSS class on
- (also: .remove( ) and .toggle( ))
- createElement → make a new element
- appendChild → place it on the page

What you will see

The paragraph turns yellow,
and "New item" appears
in the list below it.

Events — The Click



button.html

```
<!DOCTYPE html>
<html>
<body>
  <button id="btn">Click me</button>

  <script>
    const btn =
      document.getElementById("btn");

    btn.addEventListener("click", () => {
      alert("You clicked the button!");
    });
  </script>
</body>
</html>
```

- An event = something the user does
- addEventListener = "when X happens, run this"
- **Formula: find → listen → react**
- This pattern is 90% of interactive pages

What you will see

A button appears.

Click it → popup says:

"You clicked the button!"

Events — Live Typing

typing.html

```
<!DOCTYPE html>
<html>
<body>
  <input id="box" placeholder="Type here">
  <h2 id="show"></h2>

  <script>
    const box = document.getElementById("box");
    const show = document.getElementById("show");

    box.addEventListener("input", () => {
      show.textContent = box.value;
    });
  </script>
</body>
</html>
```

- "input" event fires on every keystroke
- .value → what is inside the input box
- We copy it into the heading instantly
- This is how live search boxes work!

What you will see

Whatever you type in the box appears below it — letter by letter, instantly.

Events — Forms Without Reload



form.html

```
<!DOCTYPE html>
<html>
<body>
  <form id="myForm">
    <input id="name" placeholder="Your name">
    <button>Submit</button>
  </form>
  <h2 id="hello"></h2>

  <script>
    const form = document.getElementById("myForm");

    form.addEventListener("submit", (e) => {
      e.preventDefault();  // stop page reload

      const name =
        document.getElementById("name").value;
      document.getElementById("hello")
        .textContent = "Welcome, " + name + "!";
    });
  </script>
</body>
</html>
```

- Forms normally reload the page on submit
- **e.preventDefault() stops the reload**
- Then we read the value and show a message
- Every login / signup form works this way

What you will see

Type a name, press Submit →
page shows "Welcome, Rahim!"
No reload happens.

Mini Project — Click Counter

counter.html

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="count">0</h1>
  <button id="plus">+1</button>
  <button id="reset">Reset</button>
  <script>
    let count = 0;
    const show = document.getElementById("count");

    document.getElementById("plus")
      .addEventListener("click", () => {
        count++;
        show.textContent = count;
      });

    document.getElementById("reset")
      .addEventListener("click", () => {
        count = 0;
        show.textContent = count;
      });
  </script>
</body>
</html>
```

- **Your first real app — 20 lines!**
- A variable + the DOM + two events
- Challenge: add a −1 button
- Challenge 2: turn red when count > 10

What you will see

A number with two buttons.

+1 raises it, Reset makes it 0.

You built an app!

PART 7 OF 8

Modern JS (ES6)

Template literals · Destructuring · Spread · Rest · Classes · Modules

Template Literals — Smart Text



template.js

```
const name = "Rahim";
const price = 500;

// Old way — messy plus signs:
console.log("Hello " + name + "!");

// New way — backticks ` ` :
console.log(`Hello ${name}!`);
console.log(`Total: ${price * 2} taka`);

// Multi-line? Just press Enter:
console.log(`Dear ${name},
Your bill is ${price} taka.`);
```

- Use backticks ` — not normal quotes
- ${ } = a window for any variable or math
- Multi-line text works naturally
- No more + + + glue — much cleaner

Output

```
Hello Rahim!
Hello Rahim!
Total: 1000 taka
Dear Rahim,
Your bill is 500 taka.
```

Destructuring — Unpack Fast



destructure.js

```
// From an object:
const student = {
  name: "Ayesha", dept: "CSE", age: 21
};

const { name, dept } = student;
console.log(name, dept);

// From an array (by position):
const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first, second);
```

- Pull values out of objects in one line
- { name } grabs student.name automatically
- Arrays unpack by position: [a, b]
- You'll see this in every React tutorial

Output

```
Ayesha CSE
red green
```

Spread ... — Copy & Combine



spread.js

```
const a = [1, 2, 3];
const b = [4, 5];

// Join two lists:
const joined = [...a, ...b];
console.log(joined);

// Safe copy (original untouched):
const copy = [...a];
copy.push(99);
console.log(copy);
console.log(a);

// Works on objects too:
const user = { name: "Rahim" };
console.log({ ...user, city: "Dhaka" });
```

- ... spreads items out of a list or object
- Join arrays without any loop
- Copy safely — the original stays safe
- Add fields to an object copy in one line

Output

```
[1, 2, 3, 4, 5]
[1, 2, 3, 99]
[1, 2, 3]
{name: "Rahim",
 city: "Dhaka"}
```

Rest ... — Collect the Extras



rest.js

```
// ...numbers collects ALL inputs:
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) total += n;
  return total;
}

console.log(sum(1, 2));
console.log(sum(1, 2, 3, 4, 5));

function intro(first, ...others) {
  console.log("First:", first);
  console.log("Others:", others);
}
intro("a", "b", "c");
```

- Same ... symbol, opposite job of spread
- Spread = unpack out, Rest = pack in
- sum() now takes ANY number of inputs
- The rest lands in a normal array

Output

```
3
15
First: a
Others: ["b", "c"]
```

Classes — A Factory for Objects



classes.js

```
class Student {
  constructor(name, dept) {
    this.name = name;
    this.dept = dept;
  }

  introduce() {
    console.log(
      `I am ${this.name} from ${this.dept}`);
  }
}

const s1 = new Student("Rahim", "CSE");
const s2 = new Student("Ayesha", "EEE");
s1.introduce();
s2.introduce();
```

- A class = a template / factory for objects
- constructor runs when you use new
- Each new Student( ) gets its own data
- Advanced topic — recognizing it is enough today

Output

```
I am Rahim from CSE
I am Ayesha from EEE
```

Modules — Split Code into Files



math.js + main.js

```
// ---- File 1: math.js ----
export function add(a, b) {
  return a + b;
}
export const PI = 3.1416;

// ---- File 2: main.js ----
import { add, PI } from "./math.js";

console.log(add(2, 3));
console.log(PI);

// In HTML use:
// <script type="module" src="main.js">
```

- Big projects = many small files
- export → "others may use this"
- import → "bring it here"
- Needs type="module" in the script tag

Output

```
5
3.1416
```

PART 8 OF 8

Async & Errors

setTimeout · Promises · async / await · fetch · try / catch

Async — JS Doesn't Stand Waiting



order.js

```
console.log("1. Order the food");

setTimeout(() => {
  console.log("3. Food is ready!");
}, 3000);   // after 3 seconds

console.log("2. Talk with friends");

// JS does not stand at the kitchen
// and wait — it keeps working!
```

- **Watch the order: 1, 2 ... then 3!**
- setTimeout = "run this later"
- JS continues with the next line meanwhile
- This idea is called asynchronous

Output

```
1. Order the food
2. Talk with friends
(3 seconds pass...)
3. Food is ready!
```

Promises — "I Will Tell You Later"

promise.js

```
const order = new Promise((resolve, reject) => {
  const foodReady = true;

  setTimeout(() => {
    if (foodReady)
      resolve("Biryani is served!");
    else
      reject("Kitchen is closed!");
  }, 2000);
});

order
  .then(msg => console.log(msg))
  .catch(err => console.log(err));
```

- A promise = a future result
- resolve ⟶ it worked (.then runs)
- reject ⟶ it failed (.catch runs)
- Change foodReady to false — watch .catch!

Output

```
(2 seconds pass...)
Biryani is served!
```

async / await + fetch — Real Data



fetch.js

```
// Get REAL data from the internet:
async function loadUser() {
  const res = await fetch(
    "https://api.github.com/users/github");

  const data = await res.json();

  console.log(data.name);
  console.log(data.followers);
}

loadUser();
```

- fetch = ask the internet for data
- await = "pause here until it arrives"
- await only works inside async functions
- **This exact pattern powers every modern app**

Output

```
GitHub
12345   (follower count)
```

try / catch — Fail Safely



errors.js

```
try {
  sayHello();   // this function
                // does not exist!
} catch (error) {
  console.log("Problem:", error.message);
} finally {
  console.log("Always runs");
}

console.log("Program did not crash!");
```

- try ⟶ "attempt this code"
- catch ⟶ runs only if something breaks
- finally ⟶ always runs, error or not
- Without try/catch, the error stops everything

Output

```
Problem: sayHello is not defined
Always runs
Program did not crash!
```

Reading Errors Like a Pro



debug.js

```
const items = ["pen", "book"];

console.log("Checking:", items);
console.table(items);

// The 3 errors you will meet:
// SyntaxError    -> a typo in the code
// ReferenceError -> name does not exist
// TypeError      -> wrong kind of value

// The error always shows a LINE NUMBER
// Press F12 and read it slowly!
```

- **Red errors are normal — read, don't panic**
- The line number tells you where to look
- console.table → beautiful table view
- Every professional debugs this way daily

Output

```
Checking: ["pen", "book"]
(plus a neat table
 with rows 0 and 1)
```

Your Homework — 3 Projects

1. Calculator

What it does

Two input boxes + buttons
for + − × ÷

What you practice

Uses: variables, functions,
events, DOM

2. To-Do List

What it does

Type a task → press Add →
it appears in a list

What you practice

Uses: arrays, createElement,
loops, events

3. Quiz App

What it does

Show a question, check the
answer, count the score

What you practice

Uses: objects, arrays,
if/else, DOM

Start small: the Click Counter from today is already the heart of all three projects. Grow it!



A Full Day — You Made It!

Practice daily

15 minutes of code beats
3 hours of videos

Build the projects

Calculator, to-do list,
quiz app

Use the console

F12 is your best friend —
errors are teachers

Keep these slides

Every example here
runs — try them all

Questions & Discussion

Thank you! · Anwar Hossain, Founder: Tarun Soft Pvt Limited